

PREPARACION PARA LA PRACTICA EXPERIMENTAL UNIDAD I – PASO 4

JAVASCRIPT ES6+**De cero a la practica: 30 ejemplos progresivos**

Cada ejemplo construye sobre el anterior hasta llegar al código del Paso 4

Aplicaciones Web – Ingeniería de Software – UTEQ 2025-2026

Como usar este documento

Los ejemplos están numerados del 1 al 30 en orden de dificultad. Cada uno se puede probar en la **consola del navegador** (F12 → Console). Al final del documento, el estudiante habrá visto cada línea de código que necesita escribir en el Paso 4 de la práctica.

Probar cada ejemplo: abrir Chrome → F12 → Console → pegar → Enter.

Índice

1. Nivel 1: Variables y tipos de dato	1
1.1. const y let: declarar variables	1
1.2. Template literals: texto con variables	2
1.3. Objetos y arrays	2
2. Nivel 2: Funciones y arrow functions	3
2.1. Funciones clásicas y arrow functions	3
2.2. Funciones que reciben funciones (callbacks)	3
3. Nivel 3: El DOM (Document Object Model)	4
3.1. Seleccionar elementos del HTML	4
3.2. Modificar el contenido y atributos	4
3.3. Escuchar eventos	5
4. Nivel 4: Módulos ES6 (import / export)	6
4.1. export: exponer funciones de un módulo	6
4.2. import: usar funciones de otro módulo	6
5. Nivel 5: Promesas y async/await	7
5.1. Que es una Promesa	7
5.2. async/await: Promesas legibles	8
6. Nivel 6: fetch – hablar con el servidor	8
6.1. GET: obtener datos	8
6.2. POST: enviar datos	9
7. Nivel 7: try/catch/finally – manejar errores	9
8. Nivel 8: Manipular el DOM con datos de la API	10
8.1. Renderizar datos como HTML	10

9. Nivel 9: localStorage y modo oscuro	11
10. Nivel 10: Formularios con JavaScript	12
11. Nivel FINAL: Todo junto – el código de la practica	13

Nivel 1: Variables y tipos de dato

1.1 const y let: declarar variables

Concepto

`const` declara una variable cuya referencia no se puede reasignar. `let` permite reasignar. `var` es obsoleto: no usarlo. Regla: usar siempre `const`; solo `let` si necesitas cambiar el valor después.

Ejemplo 1 – const vs let

```
1 // const: no se puede reasignar
2 const nombre = "Maria";
3 const edad = 22;
4 const esEstudiante = true;
5
6 // let: si se puede reasignar
7 let contador = 0;
8 contador = 1; // OK
9 contador = 2; // OK
10
11 // Esto daría ERROR:
12 // nombre = "Juan"; // TypeError: Assignment to constant variable
13
14 console.log(nombre, edad, esEstudiante, contador);
15 // Maria 22 true 2
```

Resultado en consola

Maria 22 true 2

1.2 Template literals: texto con variables

Ejemplo 2 – Template literals con backticks

```
1 const nombre = "Carlos";
2 const edad = 20;
3
4 // ANTES (concatenacion fea):
5 const msgViejo = "Hola, " + nombre + ". Tienes " + edad + " anos."
6 ;
7
8 // AHORA (template literal con backticks):
9 const msg = `Hola, ${nombre}. Tienes ${edad} anos.`;
```

```
9
10 // Se puede poner cualquier expresion dentro de ${}:
11 const msg2 = 'En 5 anos tendras ${edad + 5} anos.';
12
13 console.log(msg); // Hola, Carlos. Tienes 20 anos.
14 console.log(msg2); // En 5 anos tendras 25 anos.
```

Donde se usa en la practica: app.js, ui.js

Los template literals se usan en `renderizarTarjetas()` para construir el HTML de cada tarjeta, y en los mensajes de error.

1.3 Objetos y arrays

Ejemplo 3 – Objetos y arrays basicos

```
1 // Objeto: coleccion de propiedades clave:valor
2 const usuario = {
3   nombre: "Ana",
4   email: "ana@uteq.edu.ec",
5   edad: 21
6 };
7
8 // Acceder a propiedades
9 console.log(usuario.nombre); // Ana
10 console.log(usuario.email); // ana@uteq.edu.ec
11
12 // Array: lista ordenada de elementos
13 const colores = ["rojo", "verde", "azul"];
14 console.log(colores[0]); // rojo
15 console.log(colores.length); // 3
16
17 // Array de objetos (lo que devuelve una API):
18 const usuarios = [
19   { id: 1, name: "Leanne", email: "l@mail.com" },
20   { id: 2, name: "Ervin", email: "e@mail.com" },
21 ];
22 console.log(usuarios[0].name); // Leanne
```

Donde se usa en la practica: api.js

La API devuelve un array de objetos exactamente como `usuarios` del ejemplo. Cada objeto tiene propiedades como `id`, `name`, `email`.

Nivel 2: Funciones y arrow functions

2.1 Funciones classicas y arrow functions

Ejemplo 4 – Tres formas de escribir una funcion

```
1 // Forma 1: funcion clasica (funciona, pero es verbosa)
2 function sumar(a, b) {
3     return a + b;
4 }
5
6 // Forma 2: arrow function con cuerpo
7 const restar = (a, b) => {
8     return a - b;
9 };
10
11 // Forma 3: arrow function corta (1 expresion = return implicito)
12 const multiplicar = (a, b) => a * b;
13
14 console.log(sumar(3, 5)); // 8
15 console.log(restar(10, 4)); // 6
16 console.log(multiplicar(3, 7)); // 21
```

2.2 Funciones que reciben funciones (callbacks)

Ejemplo 5 – forEach, map y filter con arrow functions

```
1 const numeros = [1, 2, 3, 4, 5];
2
3 // forEach: ejecuta algo por cada elemento (no devuelve nada)
4 numeros.forEach(n => console.log(n * 2));
5 // 2, 4, 6, 8, 10
6
7 // map: transforma cada elemento y devuelve un nuevo array
8 const dobles = numeros.map(n => n * 2);
9 console.log(dobles); // [2, 4, 6, 8, 10]
10
11 // filter: filtra y devuelve solo los que cumplen la condicion
12 const pares = numeros.filter(n => n % 2 === 0);
13 console.log(pares); // [2, 4]
14
15 // Encadenado: filtrar pares y multiplicar por 10
16 const resultado = numeros
17 .filter(n => n % 2 === 0)
18 .map(n => n * 10);
19 console.log(resultado); // [20, 40]
```

Donde se usa en la practica: ui.js

En `renderizarTarjetas()` se usa `datos.map()` para transformar cada objeto en HTML, y `.join("")` para unir los fragmentos en un solo string.

Nivel 3: El DOM (Document Object Model)

3.1 Seleccionar elementos del HTML

Ejemplo 6 – getElementById y querySelector

```
1 // getElementById: busca por id (el mas rapido)
2 const titulo = document.getElementById('titulo-datos');
3 console.log(titulo); // <h2 id="titulo-datos">...</h2>
4 console.log(titulo.textContent); // el texto dentro del h2
5
6 // querySelector: busca con selector CSS (mas flexible)
7 const nav = document.querySelector('nav');
8 const primerEnlace = document.querySelector('nav a');
9 const btnTema = document.querySelector('#btn-tema');
10
11 // querySelectorAll: devuelve TODOS los que coinciden
12 const todosLosLinks = document.querySelectorAll('nav a');
13 console.log(todosLosLinks.length); // numero de enlaces
```

3.2 Modificar el contenido y atributos

Ejemplo 7 – Cambiar texto, HTML y atributos

```
1 const el = document.getElementById('estado-carga');
2
3 // textContent: cambia solo el texto (SEGURO contra XSS)
4 el.textContent = 'Cargando datos...';
5
6 // innerHTML: cambia el HTML interno (PELIGROSO si viene del
7 // usuario)
8 el.innerHTML = '<strong>Cargando...</strong>';
9
10 // setAttribute: cambia cualquier atributo
11 const btn = document.getElementById('btn-tema');
12 btn.setAttribute('aria-pressed', 'true');
13 btn.textContent = 'Modo claro';
14
15 // Leer un atributo
16 const valor = btn.getAttribute('aria-pressed'); // "true"
```

Donde se usa en la practica: ui.js

mostrarCarga() usa textContent para mostrar el mensaje. renderizarTarjetas() usa innerHTML pero con escapeHTML() para prevenir XSS. toggleTema() usa setAttribute para cambiar aria-pressed.

3.3 Escuchar eventos

Ejemplo 8 – addEventListener

```
1 // Escuchar clic en un boton
2 const btn = document.getElementById('btn-tema');
3 btn.addEventListener('click', () => {
4   console.log('Boton clickeado!');
5 });
6
7 // Escuchar cuando el DOM esta listo (ANTES de acceder a elementos
8 // )
9 document.addEventListener('DOMContentLoaded', () => {
10   console.log('DOM listo. Ahora puedo buscar elementos.');
```

11 const titulo = document.getElementById('titulo-datos');

12 // Aqui titulo NUNCA es null

13 });

14

15 // Escuchar envio de formulario

16 const form = document.getElementById('formulario-pfc');

17 form.addEventListener('submit', (e) => {

18 e.preventDefault(); // Evitar que la pagina se recargue

19 console.log('Formulario enviado sin recargar!');

20 });

Por que DOMContentLoaded

Si el `<script>` esta en el `<head>`, se ejecuta ANTES de que el HTML exista. `getElementById` devolveria `null` porque el elemento aun no existe. `DOMContentLoaded` espera a que todo el HTML este parseado. Alternativa: poner el `<script>` al final del `<body>` con `type="module"` (que es `defer` automaticamente).

Donde se usa en la practica: app.js

Todo el codigo de `app.js` esta dentro de `document.addEventListener('DOMContentLoaded', async () =>{...})`. El listener del formulario usa `e.preventDefault()` para evitar la recarga.

Nivel 4: Modulos ES6 (import / export)

4.1 export: exponer funciones de un modulo

Ejemplo 9 – export en un modulo

```
1 // archivo: js/modules/matematicas.js
2
3 // Export individual (named export)
4 export function sumar(a, b) {
5   return a + b;
6 }
7
```

```
8   export function restar(a, b) {
9       return a - b;
10  }
11
12  // Variable exportada
13  export const PI = 3.14159;
14
15  // Funcion privada (sin export = no se puede importar)
16  function logInterno(msg) {
17      console.log('[mate] ${msg}');
18  }
```

4.2 import: usar funciones de otro modulo

Ejemplo 10 – import en el archivo principal

```
1   // archivo: js/app.js
2
3   // Importar funciones especificas
4   import { sumar, restar, PI } from './modules/matematicas.js';
5
6   console.log(sumar(3, 5)); // 8
7   console.log(PI);          // 3.14159
8
9   // NO se puede acceder a logInterno() porque no tiene export
```

Requisito en el HTML

Para que import/export funcionen, el script debe tener type="module":

```
<script type="module"src="js/app.js"></script>
```

Sin type="module" el navegador lanza: SyntaxError: Cannot use import statement outside a module.

Donde se usa en la practica: Toda la arquitectura del Paso 4

El PFC tiene 3 modulos: api.js exporta obtenerDatos() y enviarDatos(). ui.js exporta mostrarCarga(), renderizarTarjetas(), escapeHTML(), toggleTema(). app.js importa todo y orquesta.

Nivel 5: Promesas y async/await

5.1 Que es una Promesa

Concepto

Una **Promesa** es un objeto que representa un valor que **todavía no existe** pero existira en el futuro (o fallara). Tiene 3 estados: **pending** (esperando), **fulfilled** (exito) y **rejected** (error). `fetch()` devuelve una Promesa.

Ejemplo 11 – Promesa basica con `.then()/catch()`

```
1 // fetch() devuelve una Promesa
2 // .then() se ejecuta cuando la Promesa se resuelve (exito)
3 // .catch() se ejecuta cuando la Promesa se rechaza (error)
4
5 fetch('https://jsonplaceholder.typicode.com/users/1')
6   .then(respuesta => respuesta.json())
7   .then(usuario => console.log(usuario.name))
8   .catch(error => console.error('Fallo:', error));
9
10 // Resultado: "Leanne Graham"
```

5.2 async/await: Promesas legibles

Ejemplo 12 – Lo mismo pero con `async/await`

```
1 // async hace que la funcion devuelva una Promesa
2 // await pausa la ejecucion hasta que la Promesa se resuelva
3 // SOLO se puede usar await dentro de una funcion async
4
5 async function cargarUsuario() {
6   const respuesta = await fetch(
7     'https://jsonplaceholder.typicode.com/users/1'
8   );
9   const usuario = await respuesta.json();
10  console.log(usuario.name); // "Leanne Graham"
11 }
12
13 cargarUsuario();
```

Por que `async/await` es mejor que `.then()`

Comparar los ejemplos 11 y 12: hacen exactamente lo mismo, pero el ejemplo 12 se lee como codigo normal de arriba a abajo. Con `.then()` encadenado, el codigo se vuelve dificil de leer cuando hay 3 o 4 operaciones asincronas seguidas (“callback hell”).

Nivel 6: fetch – hablar con el servidor

6.1 GET: obtener datos

Ejemplo 13 – fetch GET con verificación de error

```
1  async function obtenerUsuarios() {
2    const res = await fetch(
3      'https://jsonplaceholder.typicode.com/users'
4    );
5
6    // IMPORTANTE: fetch NO lanza error con 404 o 500
7    // Solo lanza error si no hay internet o el dominio no existe
8    // Por eso verificamos manualmente:
9    if (!res.ok) {
10     throw new Error('Error HTTP ${res.status}');
11   }
12
13   const datos = await res.json(); // Convertir respuesta a objeto
14   JS
15   console.log('Recibidos ${datos.length} usuarios');
16   return datos;
17 }
18
19 obtenerUsuarios();
20 // Recibidos 10 usuarios
```

Donde se usa en la practica: `api.js` — función `obtenerDatos()`

Este es exactamente el patron de la función `obtenerDatos()` del modulo `api.js` de la practica.

6.2 POST: enviar datos

Ejemplo 14 – fetch POST con JSON

```
1  async function crearUsuario(datos) {
2    const res = await fetch(
3      'https://jsonplaceholder.typicode.com/users',
4      {
5        method: 'POST', // Verbo HTTP
6        headers: { 'Content-Type': 'application/json' }, // Tipo
7        body: JSON.stringify(datos), // Objeto -> texto
8        JSON
9      }
10   );
11
12   if (!res.ok) {
13     throw new Error('Error al crear: ${res.status}');
14   }
15
16   return res.json();
17 }
18
19 // Usar:
20 crearUsuario({ nombre: "Ana", email: "ana@uteq.edu.ec" })
```

```
20 .then(resp => console.log('Creado:', resp));
```

Donde se usa en la practica: `api.js` — funcion `enviarDatos()`

Este es el patron de `enviarDatos()` que se usa cuando el formulario se envia.

Nivel 7: try/catch/finally – manejar errores

Ejemplo 15 – Los 4 estados que siempre hay que manejar

```
1  async function cargarDatos() {
2    // ESTADO 1: Cargando
3    console.log('Cargando...');
4
5    try {
6      const res = await fetch(
7        'https://jsonplaceholder.typicode.com/users?_limit=3'
8      );
9      if (!res.ok) throw new Error('HTTP ${res.status}');
10     const datos = await res.json();
11
12     // ESTADO 2: Exito
13     if (datos.length === 0) {
14       // ESTADO 3: Vacio (exito pero sin datos)
15       console.log('No hay datos.');
```

Resultado esperado

Cargando...

Datos: ["Leanne Graham", "Ervin Howell", "Clementine Bauch"]

Fin. Ocultar indicador de carga.

Donde se usa en la practica: app.js — funcion cargarDatos()

Este patron de 4 estados es exactamente la funcion `cargarDatos()` de `app.js`. La practica usa `mostrarCarga()` en lugar de `console.log` y `renderizarTarjetas()` en lugar de `console.log` del array.

Nivel 8: Manipular el DOM con datos de la API

8.1 Renderizar datos como HTML

Ejemplo 16 – Convertir un array de objetos en tarjetas HTML

```
1      // Funcion anti-XSS: OBLIGATORIA antes de insertar texto en
      innerHTML
2      function escapeHTML(texto) {
3          return String(texto)
4              .replace(/&/g, '&amp;')
5              .replace(/</g, '&lt;')
6              .replace(/>/g, '&gt;')
7              .replace(/"/g, '&quot;');
8      }
9
10     // Datos que vinieron de la API:
11     const usuarios = [
12         { id: 1, name: "Leanne Graham", email: "l@mail.com" },
13         { id: 2, name: "Ervin Howell", email: "e@mail.com" },
14     ];
15
16     // Convertir cada objeto en HTML con .map() + template literal
17     const html = usuarios.map(u => `
18     <article class="tarjeta">
19     <h3>${escapeHTML(u.name)}</h3>
20     <p>${escapeHTML(u.email)}</p>
21     <p><small>ID: ${u.id}</small></p>
22     </article>
23     `).join(''); // .join('') une los fragmentos en 1 solo string
24
25     // Insertar en el DOM
26     const contenedor = document.getElementById('contenedor-datos');
27     contenedor.innerHTML = html;
```

Por que escapeHTML es obligatorio

Si un usuario malicioso guardara como nombre `<script>alert('XSS')</script>`, sin `escapeHTML` ese script se ejecutaria en el navegador de todos los demas usuarios. Con `escapeHTML`, los `<` y `>` se convierten en `<` y `>`, y el navegador muestra el texto en lugar de ejecutarlo.

Donde se usa en la practica: ui.js — funcion renderizarTarjetas()

Este ejemplo es exactamente lo que hace renderizarTarjetas() en el Paso 4.

Nivel 9: localStorage y modo oscuro

Ejemplo 17 – localStorage: guardar y leer datos persistentes

```
1 // Guardar un valor (persiste al cerrar el navegador)
2 localStorage.setItem('tema-pfc', 'oscuro');
3
4 // Leer un valor (devuelve null si no existe)
5 const tema = localStorage.getItem('tema-pfc');
6 console.log(tema); // "oscuro"
7
8 // Eliminar un valor
9 localStorage.removeItem('tema-pfc');
```

Ejemplo 18 – Toggle de modo oscuro completo

```
1 function toggleTema() {
2   const html = document.documentElement; // = <html>
3   const actual = html.getAttribute('data-tema');
4   const nuevo = actual === 'oscuro' ? 'claro' : 'oscuro';
5
6   // Cambiar el atributo data-tema en <html>
7   html.setAttribute('data-tema', nuevo);
8
9   // Guardar la preferencia
10  localStorage.setItem('tema-pfc', nuevo);
11
12  // Actualizar el texto del boton
13  const btn = document.getElementById('btn-tema');
14  btn.textContent = nuevo === 'oscuro' ? 'Modo claro' : 'Modo
15    oscuro';
16  btn.setAttribute('aria-pressed', nuevo === 'oscuro');
17 }
18
19 // Al cargar la pagina: restaurar la preferencia guardada
20 function cargarTema() {
21   const guardado = localStorage.getItem('tema-pfc');
22   if (guardado) {
23     document.documentElement.setAttribute('data-tema', guardado);
24   }
25 }
```

Donde se usa en la practica: ui.js — funciones toggleTema() y cargarTema()

Estas dos funciones son exactamente las que van en ui.js. El CSS en variables.css tiene la regla [data-tema=."oscuro"] que cambia los colores automaticamente cuando el atributo cambia.

Nivel 10: Formularios con JavaScript

Ejemplo 19 – Validar y enviar un formulario sin recargar

```
1  function configurarFormulario() {
2    const form = document.getElementById('formulario-pfc');
3    if (!form) return; // Proteccion si el formulario no existe
4
5    form.addEventListener('submit', async (e) => {
6      // 1. Evitar que la pagina se recargue
7      e.preventDefault();
8
9      // 2. Verificar validacion nativa del HTML
10     if (!form.checkValidity()) {
11       form.reportValidity(); // Muestra los mensajes del navegador
12       return; // No continuar si hay errores
13     }
14
15     // 3. Extraer los datos del formulario como objeto
16     const datos = Object.fromEntries(new FormData(form));
17     console.log('Datos del formulario:', datos);
18     // { nombre: "Ana", email: "ana@...", carrera: "sw", ... }
19
20     // 4. Enviar a la API
21     try {
22       const resp = await enviarDatos('users', datos);
23       console.log('Enviado:', resp);
24       form.reset(); // Limpiar el formulario
25
26       // 5. Mostrar confirmacion accesible
27       const estado = document.getElementById('estado-carga');
28       estado.textContent = 'Registro exitoso.';
29       setTimeout(() => { estado.textContent = ''; }, 4000);
30     } catch (error) {
31       console.error('Error al enviar:', error);
32     }
33   });
34 }
```

Object.fromEntries + FormData

new FormData(form) captura todos los campos del formulario. Object.fromEntries() convierte esos pares clave-valor en un objeto JavaScript plano, listo para enviar como JSON con JSON.stringify().

Donde se usa en la practica: app.js — funcion configurarFormulario()

Esta funcion es exactamente la que va en `app.js`. Conecta el formulario HTML con la API sin recargar la pagina.

Nivel FINAL: Todo junto – el codigo de la practica**Verificacion final**

Si entendiste los 19 ejemplos anteriores, puedes leer y escribir cada linea del Paso 4 de la practica. El codigo final es la **combinacion** de todos los conceptos anteriores:

#	Concepto	Donde se usa en la practica	Ejemplo en este documento
1	<code>const / let</code>	Todas las variables	Ejemplo 1
2	Template literals	HTML de tarjetas, mensajes de error	Ejemplo 2
3	Objetos y arrays	Datos de la API	Ejemplo 3
4	Arrow functions	Callbacks de <code>map</code> , <code>forEach</code> , <code>listeners</code>	Ejemplo 4
5	<code>.map()</code> y <code>.join()</code>	<code>renderizarTarjetas()</code>	Ejemplo 5
6	<code>getElementById</code>	Acceder a elementos del DOM	Ejemplo 6
7	<code>textContent / innerHTML</code>	<code>mostrarCarga()</code> , <code>renderizar()</code>	Ejemplo 7
8	<code>addEventListener</code>	Click del boton, submit del form	Ejemplo 8
9	<code>DOMContentLoaded</code>	Punto de entrada de <code>app.js</code>	Ejemplo 8
10	<code>export / import</code>	Arquitectura de 3 modulos	Ejemplos 9 y 10
11	<code>async / await</code>	<code>obtenerDatos()</code> , <code>cargarDatos()</code>	Ejemplo 12
12	<code>fetch()</code> GET	<code>obtenerDatos()</code>	Ejemplo 13
13	<code>fetch()</code> POST	<code>enviarDatos()</code>	Ejemplo 14
14	<code>try/catch/finally</code>	4 estados de carga	Ejemplo 15
15	<code>escapeHTML()</code>	Prevencion XSS	Ejemplo 16
16	<code>.map().join() + HTML</code>	<code>renderizarTarjetas()</code>	Ejemplo 16
17	<code>localStorage</code>	Persistir el tema oscuro	Ejemplo 17
18	Toggle de tema	<code>toggleTema()</code> / <code>cargarTema()</code>	Ejemplo 18
19	<code>FormData + preventDefault</code>	<code>configurarFormulario()</code>	Ejemplo 19

Estas listo

Si puedes explicar cada fila de esta tabla, puedes implementar el Paso 4 completo de la practica sin copiar y pegar. Abre la guia de la PE U1, ve al Paso 4, y veras que cada linea de codigo corresponde a uno de estos 19 ejemplos.